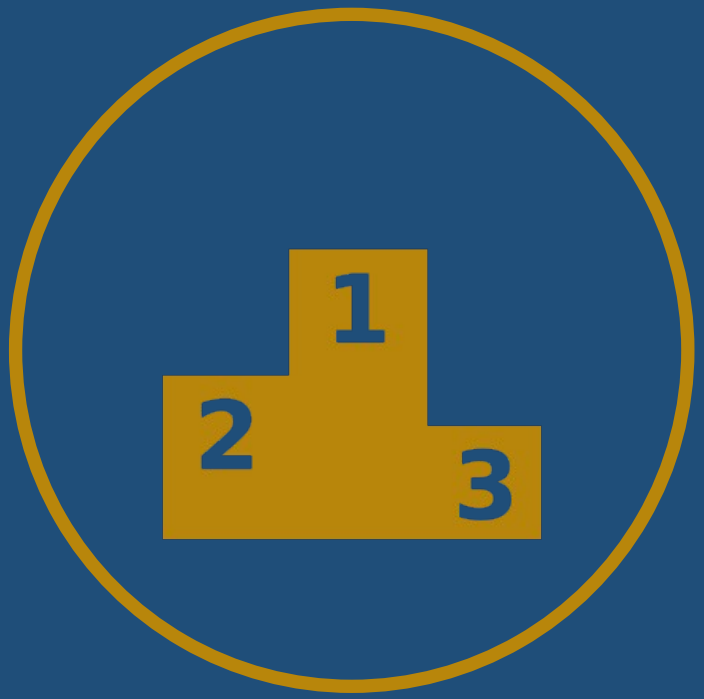
INFORMATIK LK · WELT 2 · DS 13–17

Die Rangliste

Suchen, Sortieren, Effizienz



L2.1 ····· L2.2 ····· DUELL ····· L2.3 ····· BOSS

Welt 2 im Überblick



DS 13

IDE-Level 2.1

Bausteine

Maximum, Zählen, lineare Suche, Duplikate

IDE + Lab 2.1



DS 14

IDE-Level 2.2

Sortieren

Selection-, Insertion-, Bubblesort, sortiert einfügen

IDE + Lab 2.2



DS 15

Sortier-Duell

Effizienz

Vergleiche zählen, bester und schlechtester Fall

Team-Duell + Lab 2.3



DS 16

IDE-Level 2.3

Binäre Suche

lo, hi, mid: halbieren statt durchlaufen

IDE 2.3 + Lab 2.4



DS 17

Boss 2

Das Turnier

Abi auf Zeit: ≈ 24 BE in 60 min

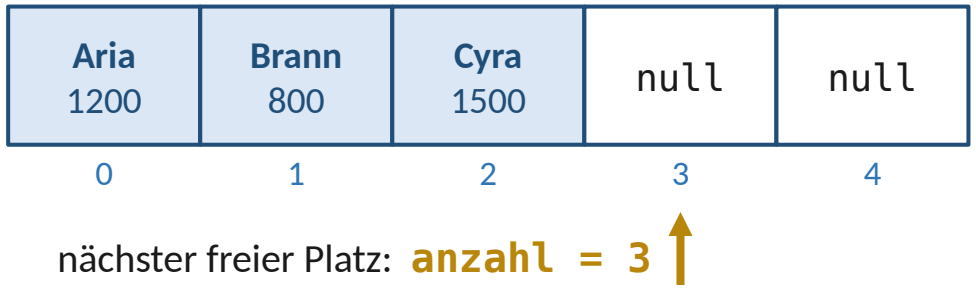
Papier + Online-Boss

Die Highscore-Liste: Eintrag und Rangliste

Eintrag
- name: Zeichenkette - punkte: Ganzzahl - zeit: Ganzzahl
+ c Eintrag(name: Zeichenkette, punkte: Ganzzahl, zeit: Ganzzahl) + getName(): Zeichenkette + getPunkte(): Ganzzahl + getZeit(): Ganzzahl + beschreibung(): Zeichenkette

Rangliste
- eintraege: Reihung von Eintrag - anzahl: Ganzzahl - vergleiche: Ganzzahl - vertauschungen: Ganzzahl
+ c Rangliste(kapazitaet: Ganzzahl) + getAnzahl(): Ganzzahl + getEintrag(i: Ganzzahl): Eintrag + hinzufuegen(e: Eintrag): Wahrheitswert - besser(a: Eintrag, b: Eintrag): Wahrheitswert + vertausche(i: Ganzzahl, j: Ganzzahl): nichts + hoechstePunktzahl(): Ganzzahl + selectionSort(): nichts …

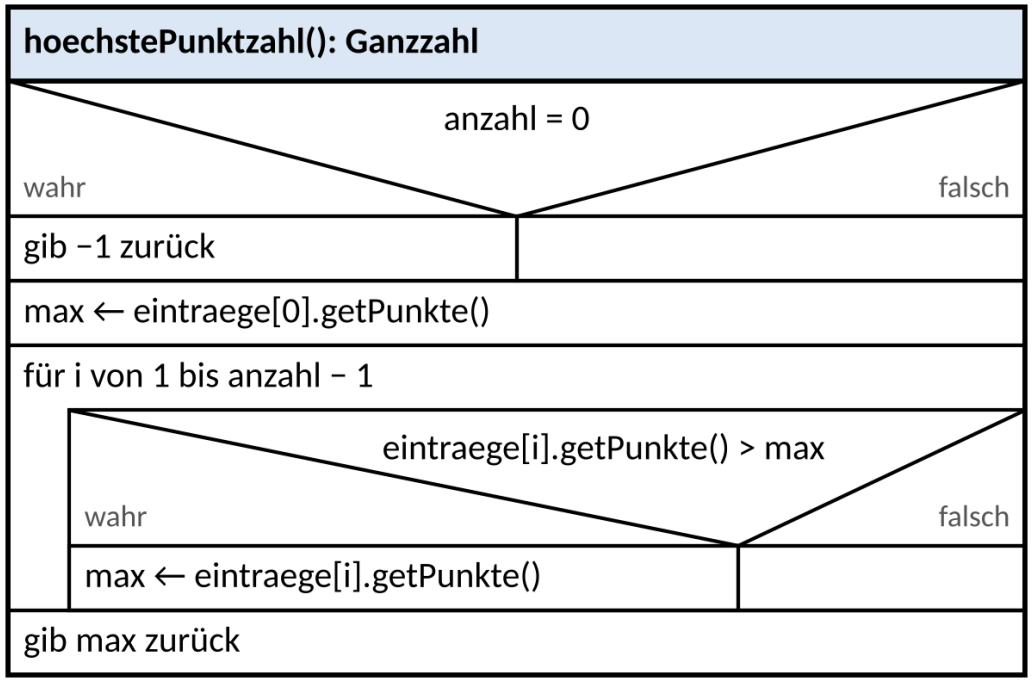
`eintraege` (Kapazität 5)



Sortiert wird absteigend: Platz 1 = meiste Punkte. **`besser`** zählt jeden Vergleich, **`vertausche`** jede Vertauschung.

Baustein 1: Maximum finden

```java
public int hoechstePunktzahl() {
    if (anzahl == 0) {
        return -1;
    }
    int max = eintraege[0].getPunkte();
    for (int i = 1; i < anzahl; i++) {
        if (eintraege[i].getPunkte() > max) {
            max = eintraege[i].getPunkte();
        }
    }
    return max;
}
```



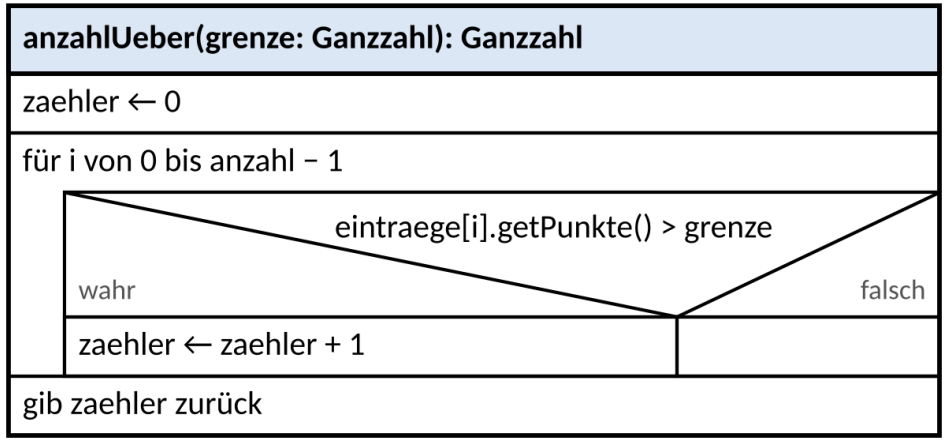
dasselbe als Struktogramm (EH §5)

Achtung: nicht mit `max = 0` starten – Strafpunkte können negativ sein. Mit dem ersten Eintrag beginnen.

besterEintrag(): gleiche Idee, aber den Eintrag merken. Mit **>** statt **>=** bleibt bei Gleichstand der erste.

Baustein 2: Zählen und Durchschnitt

```java
public int anzahlUeber(int grenze) {
    int zaehler = 0;
    for (int i = 0; i < anzahl; i++) {
        if (eintraege[i].getPunkte() > grenze) {
            zaehler++;
        }
    }
    return zaehler;
}
```




```java
public double durchschnitt() {
    if (anzahl == 0) {
        return 0.0;
    }
    int summe = 0;
    for (int i = 0; i < anzahl; i++) {
        summe = summe + eintraege[i].getPunkte();
    }
    return (double) summe / anzahl;
}
```

Achtung: `(double)` vor der Division – sonst rechnet Java ganzzahlig: 7 / 2 = 3.

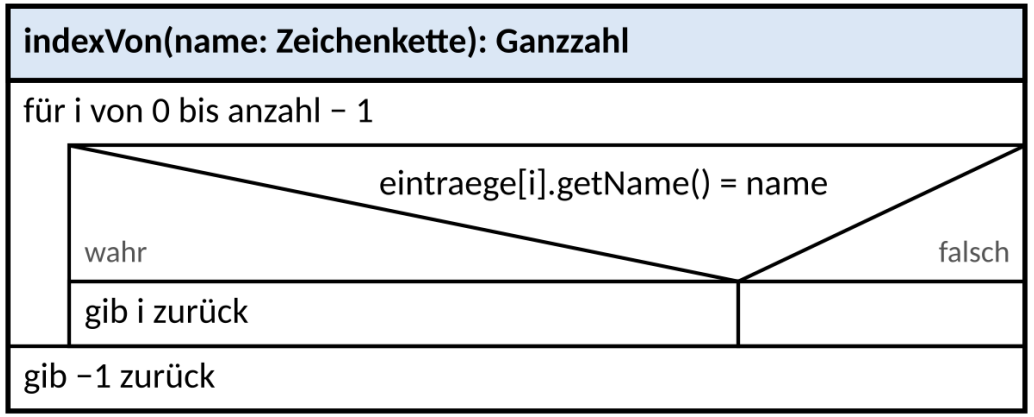
Liste 50, 80, 30, 90, 70

`anzahlUeber(60)` → **3**

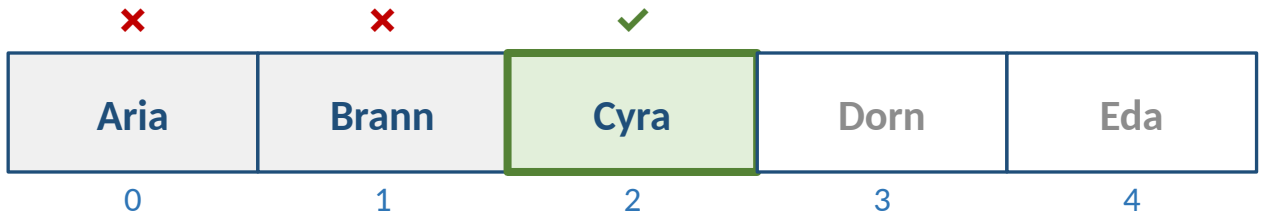
`durchschnitt()` → **320 / 5 = 64.0**

Baustein 3: Lineare Suche

```java
public int indexVon(String name) {
    for (int i = 0; i < anzahl; i++) {
        if (eintraege[i].getName().equals(name)) {
            return i;
        }
    }
    return -1;
}
```



`indexVon("Cyra")` prüft von vorn, bis zum ersten Treffer



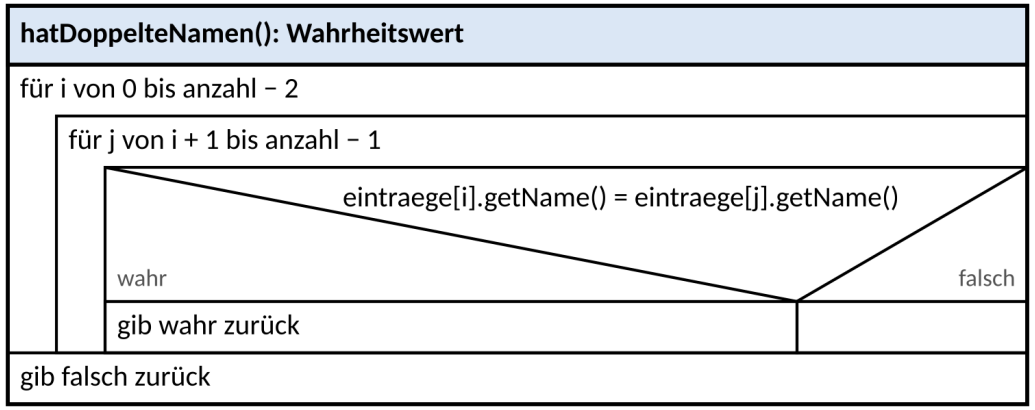
→ **gib 2 zurück**
nach 3 Prüfungen

Achtung Abi: Zeichenketten mit **`equals`** vergleichen, nie mit **==**. Die Online-IDE verzeiht ==, echtes Java nicht.

Merke: **`return -1`** steht nach der Schleife – nicht im else. Sonst wird nur der erste Eintrag geprüft.

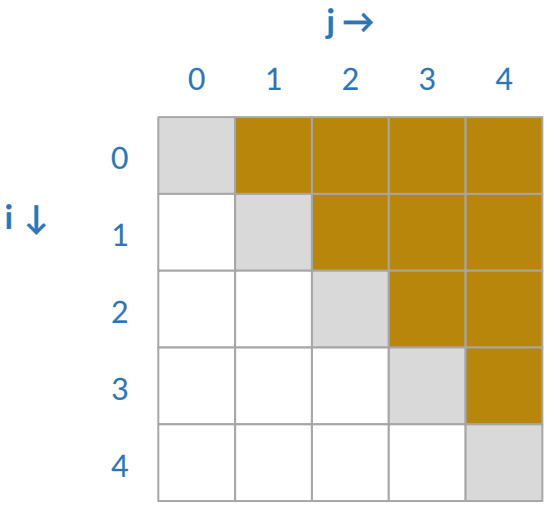
Baustein 4: Duplikate – jeder mit jedem

```java
public boolean hatDoppelteNamen() {
    for (int i = 0; i < anzahl - 1; i++) {
        for (int j = i + 1; j < anzahl; j++) {
            if (eintraege[i].getName()
                .equals(eintraege[j].getName())) {
                return true;
            }
        }
    }
    return false;
}
```



Tipp: Die innere Schleife beginnt bei `i + 1`: kein Eintrag wird mit sich selbst verglichen, kein Paar doppelt.

→ Diese Dreieckszahl begegnet uns beim Sortieren wieder.



Jedes Paar einmal

n = 5: 4 + 3 + 2 + 1 = 10

Vergleiche im schlechtesten Fall

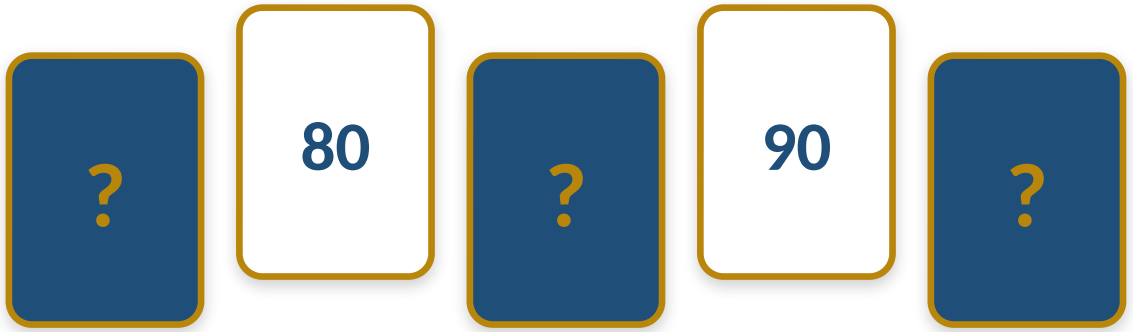
Abi-Check: Leerabfrage, equals, Grenzen

	FALSCH	RICHTIG

Leerabfrage
leere Liste: eintraege[0] ist null → Absturz
	`int max = eintraege[0]`
	`  .getPunkte();`
		`if (anzahl == 0) {`
		`  return -1;`
		`}`

Zeichenketten == vergleicht Referenzen, nicht den Inhalt	`if (e.getName() == name)`	`if (e.getName().equals(name))`
Schleifengrenze hinter anzahl − 1 steht null	`for (int i = 0; i <= anzahl; i++)`	`for (int i = 0; i < anzahl; i++)`
Rückgabe return −1 erst nach der Schleife	`if (…) { return i; }` `else { return -1; }`	`for (…) { if (…) { return i; } }` `return -1;`

Sortieren unplugged: das Karten-Turnier



verdeckt – nur zwei Karten dürfen gleichzeitig aufgedeckt werden

Schiedsrichter: deckt zwei Karten auf, sagt, welche besser ist → Strich bei Vergleiche

Tauscher: vertauscht zwei Karten → Strich bei Vertauschungen

Stratege: entscheidet, welche zwei Karten als Nächstes verglichen werden

So läuft die Runde

1. Jedes Team zieht eine Strategie-Karte: Auswahl, Einfügen oder Blasen.
2. Fünf Karten verdeckt mischen, dann absteigend sortieren: meiste Punkte links.
3. Jeder Vergleich und jede Vertauschung bekommt einen Strich.
4. Danach Karten tauschen, Strategie wechseln – wer braucht am wenigsten?

Selectionsort: das Beste nach vorn

Idee: Suche im unsortierten Rest den besten Eintrag und tausche ihn an die erste Stelle des Rests. Der sortierte Bereich wächst von links.

```
public void selectionSort() {
    for (int i = 0; i < anzahl - 1; i++) {
        int maxIndex = i;
        for (int j = i + 1; j < anzahl; j++) {
            if (besser(eintraege[j],
                       eintraege[maxIndex])) {
                maxIndex = j;
            }
        }
        if (maxIndex != i) {
            vertausche(i, maxIndex);
        }   // → Tabellenzeile
    }
}
```

Beispiel: 50, 80, 30, 90, 70 absteigend

Zustand	0	1	2	3	4	V / T
Start	50	80	30	90	70	
i = 0	90	80	30	50	70	4 / 1
i = 1	90	80	30	50	70	7 / 1
i = 2	90	80	70	50	30	9 / 2
i = 3	90	80	70	50	30	10 / 2

sortierter Bereich getauscht

10 Vergleiche · 2 Vertauschungen
Vergleiche: immer 4 + 3 + 2 + 1 – egal, wie die Daten liegen.

Insertionsort: einsortieren wie Spielkarten

Idee: Nimm den nächsten Eintrag und lass ihn nach links wandern, solange er besser ist als sein linker Nachbar.

```
public void insertionSort() {
    for (int i = 1; i < anzahl; i++) {
        int j = i;
        while (j > 0 && besser(eintraege[j],
                               eintraege[j - 1])) {
            vertausche(j, j - 1);
            j--;
        }   // → Tabellenzeile
    }
}
```

Beispiel: 50, 80, 30, 90, 70 absteigend

Zustand	0	1	2	3	4	V / T
Start	50	80	30	90	70	
i = 1	80	50	30	90	70	1 / 1
i = 2	80	50	30	90	70	2 / 1
i = 3	90	80	50	30	70	5 / 4
i = 4	90	80	70	50	30	8 / 6

sortierter Bereich eingefügter Eintrag

8 Vergleiche · 6 Vertauschungen
Abbruch beim ersten „nicht besser“ – sortierte Daten sind billig.

Bubblesort: Nachbarn tauschen

Idee: Vergleiche Nachbarn von links nach rechts und tausche, wenn der rechte besser ist. Der Schlechteste sinkt nach hinten. Abbruch, wenn ein Durchlauf ohne Tausch bleibt.

```
public void bubbleSort() {
    boolean getauscht = true;
    int durchlauf = 0;   // → Tabellenzeile (Startwerte)
    while (getauscht) {
        getauscht = false;
        for (int j = 0; j < anzahl - 1 - durchlauf; j++) {
            if (besser(eintraege[j + 1], eintraege[j])) {
                vertausche(j, j + 1);
                getauscht = true;
            }
        }
        durchlauf++;   // → Tabellenzeile
    }
}
```

Beispiel: 50, 80, 30, 90, 70 absteigend

Zustand	0	1	2	3	4	V / T
Start	50	80	30	90	70	
Lauf 1	80	50	90	70	30	4 / 3
Lauf 2	80	90	70	50	30	7 / 5
Lauf 3	90	80	70	50	30	9 / 6
Lauf 4	90	80	70	50	30	10 / 6

sortierter Bereich

10 Vergleiche · 6 Vertauschungen
Lauf 4 ohne Tausch → Abbruch. Sortierte Daten: nur 4 Vergleiche.

Drei Verfahren im Vergleich (n = 5)

	Selectionsort	Insertionsort	Bubblesort (mit Abbruch)
Idee	Bestes des Rests nach vorn tauschen	nächsten Eintrag links einsortieren	Nachbarn tauschen, Schlechtester sinkt nach hinten
Beispiel 50, 80, 30, 90, 70	**10 V · 2 T**	**8 V · 6 T**	**10 V · 6 T**
bereits sortiert	10 V · 0 T	**4 V · 0 T**	**4 V · 0 T**
umgekehrt sortiert	10 V · 2 T	**10 V · 10 T**	**10 V · 10 T**
stabil?	nein	ja	ja

V = Vergleiche (Aufrufe von besser) · T = Vertauschungen (Aufrufe von vertausche) – gezählt mit der Musterlösung

Selectionsort vergleicht immer gleich oft, tauscht aber am wenigsten.

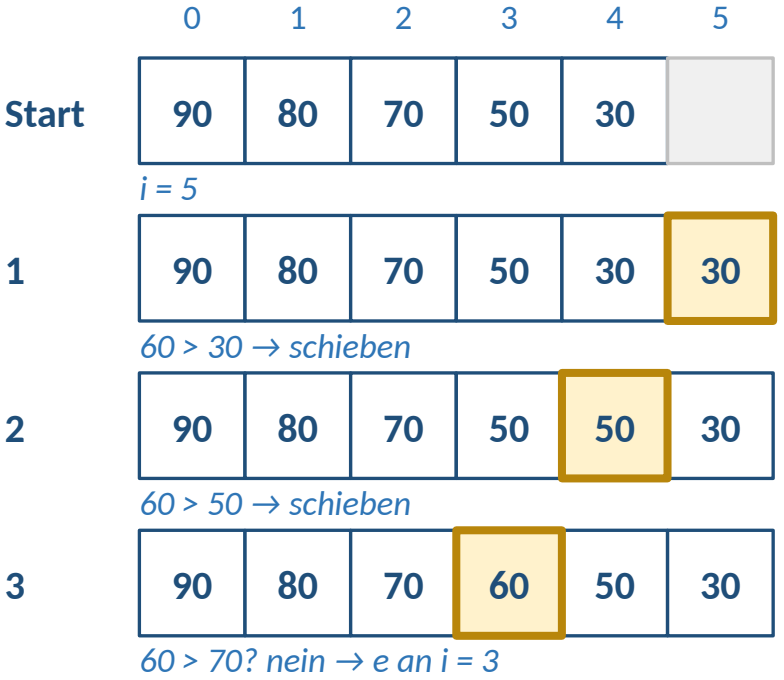
Insertion- und Bubblesort hängen von den Daten ab: bester Fall sortiert, schlechtester umgekehrt.

Sortiert einfügen: Platz schaffen von hinten

```
public boolean einfuegenSortiert(Eintrag e) {
    if (e == null || anzahl >= eintraege.length) {
        return false;
    }
    int i = anzahl;
    while (i > 0 && besser(e, eintraege[i - 1])) {
        eintraege[i] = eintraege[i - 1];
        i--;
    }
    eintraege[i] = e;
    anzahl++;
    return true;
}
```

Achtung: erst null und „voll“ prüfen – sonst greift **`eintraege[anzahl]`** hinter das Ende der Reihung.

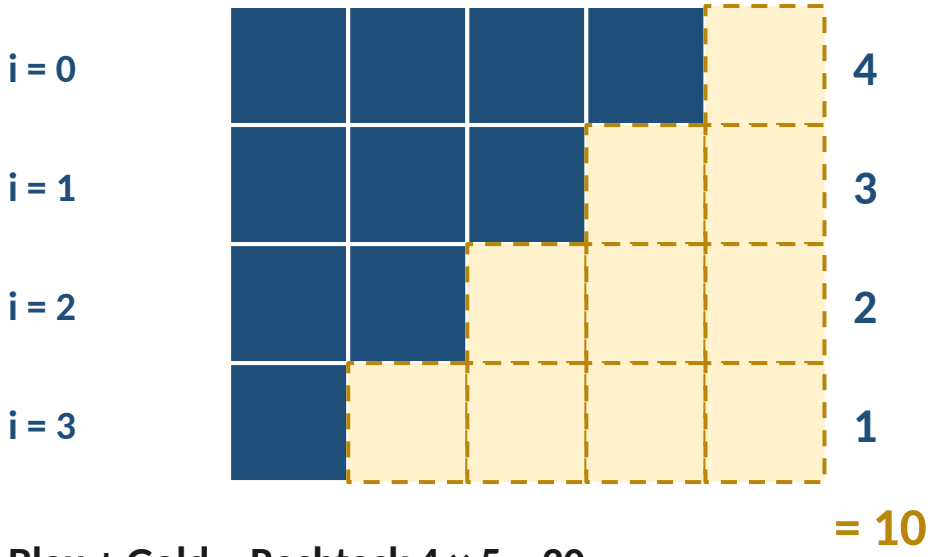
Neuer Eintrag mit 60 Punkten



3 Vergleiche · 0 Vertauschungen
Verschieben statt Tauschen. Bei Gleichstand stoppen → stabil.

Vergleiche zählen: n(n − 1) / 2

Selectionsort mit n = 5: Vergleiche je Durchlauf



Blau + Gold = Rechteck 4 × 5 = 20
Blau ist genau die Hälfte: 20 / 2 = 10

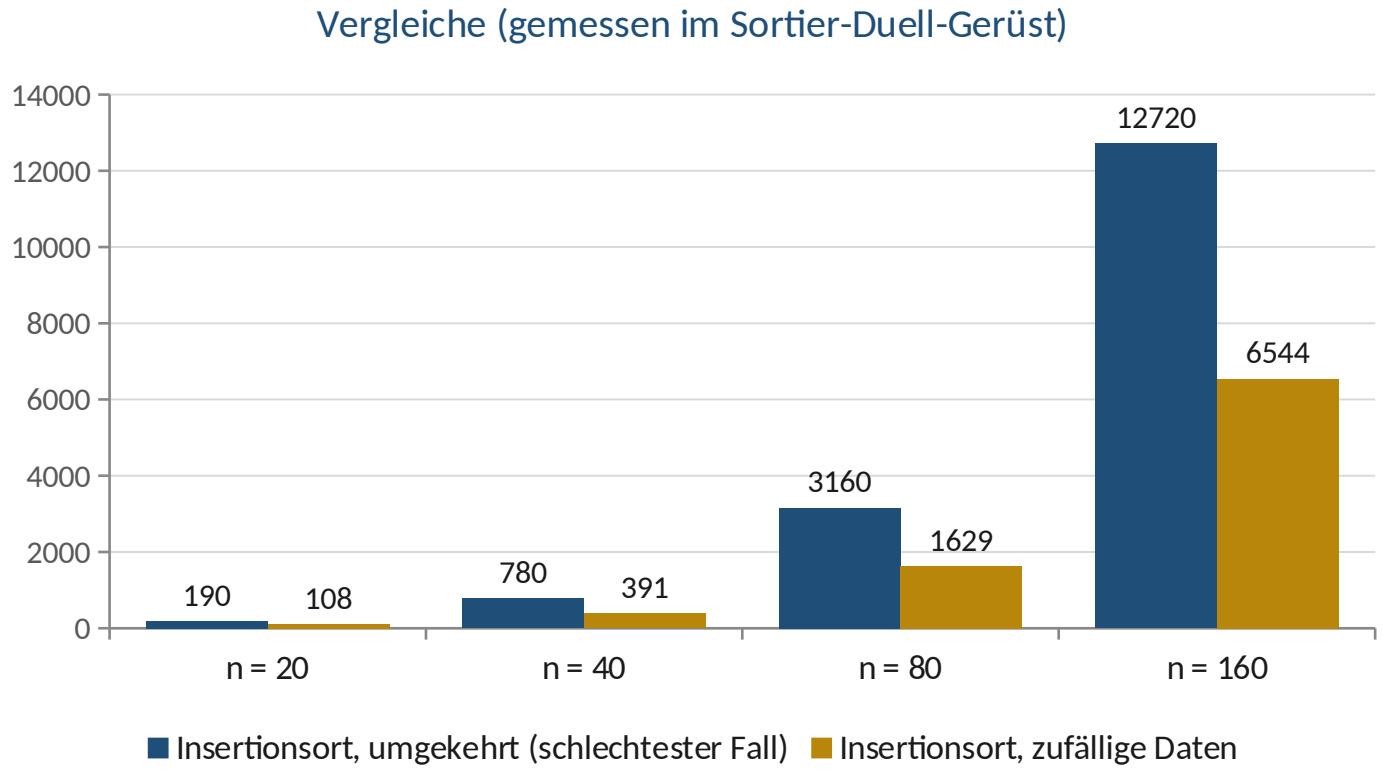
Achtung Abi: Zum schlechtesten Fall immer eine konkrete Eingabe angeben, z. B. „aufsteigend sortiert“.

(n − 1) + (n − 2) + … + 1
= n · (n − 1) / 2

n = 5 → 10 n = 40 → 780
n = 100 → 4 950

Verfahren	bester Fall	schlechtester Fall
Selection	n(n − 1)/2	n(n − 1)/2
Insertion	n − 1 (sortiert)	n(n − 1)/2 (umgekehrt)
Bubble	n − 1 (sortiert)	n(n − 1)/2 (umgekehrt)

Doppelt so viele Einträge – viermal so viel Arbeit



× 4 **schlechtester Fall**
190 → 780 → 3 160 → 12 720

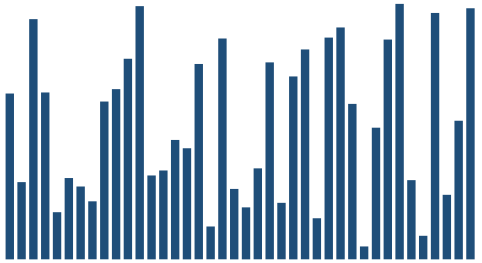
× 4 **zufällige Daten**
391 → 1 629 → 6 544

× 2 **bester Fall: n − 1**
19 → 39 → 79 → 159

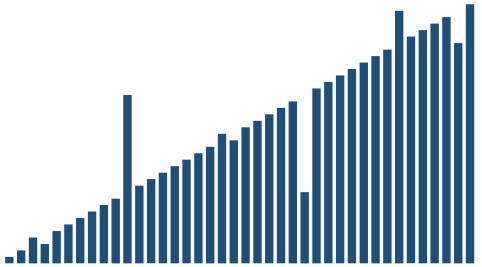
Abi-Formulierung: Verdoppelt man n, vervierfacht sich die Anzahl der Vergleiche ungefähr, da sie quadratisch von n abhängt.

Das Sortier-Duell: Wer sortiert am sparsamsten?

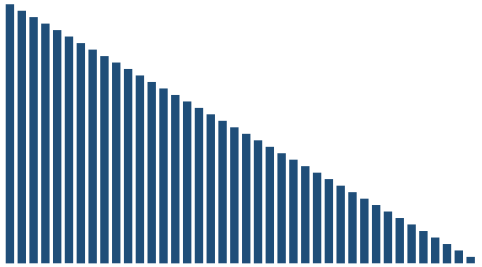
Runde 1 · zufall



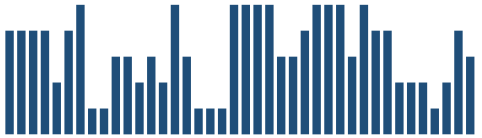
Runde 2 · fast sortiert



Runde 3 · umgekehrt



Runde 4 · viele gleiche



je 40 Balken, aufsteigend sortieren · für alle Teams exakt gleiche Daten · danach Kontrollrunde mit 37 unbekannten Werten

Erlaubt

- Vis.laenge()
- Vis.kleiner(i, j) zählt 1 Vergleich
- Vis.vertausche(i, j) zählt 1 Tausch
- eigene Variablen und Hilfsmethoden

Verboten → ungültig

- Vis.wert(i) liefert im Duell −1
- feste Tauschfolgen, 40 statt Vis.laenge()
- Vis.starten, Vis.fertig, Vis verändern

Wertung

- Summe der Vergleiche aus 4 Runden
- weniger ist besser
- Gleichstand: weniger Vertauschungen
- gültig nur, wenn alle 5 Runden sortiert

Die Marke zum Schlagen: Das Gerüst-duellSort (Insertionsort) braucht **1612 Vergleiche**. Geht es sparsamer?

Das Duell auswerten: Team-Wertung an der Tafel

Team	Vergleiche	Vertauschungen	gültig?

Vorlage für Tafel oder Beamer – Unterrichtsspiel ohne Lab-Wertung

```
DUELL-ERGEBNIS: Vergleiche = …, Vertauschungen = …, gueltig = ja
```

letzte Zeile der Konsole nach Duell.starten()

1. **Duell.starten()**
   vier Runden + Kontrollrunde, am Ende die Ergebniszeile
2. **Ein Ergebnis je Team**
   das Team entscheidet, welche Version antritt
3. **Eintrag in die Tabelle**
   Vergleiche, Vertauschungen, gültig – an der Tafel oder am Beamer
4. **Sieger**
   wenigste Vergleiche; ungültige Ergebnisse werden gestrichen



Siegerlösung vorstellen
Ruhm statt Punkte – Punkte gibt es nur im Lab

Binäre Suche: halbieren statt durchlaufen

```java
public static int binaer(int[] r, int x) {
    int lo = 0;
    int hi = r.length - 1;
    while (lo <= hi) {
        int mid = (lo + hi) / 2;  // → Tabellenzeile
        if (r[mid] == x) {
            return mid;
        }
        if (r[mid] < x) {
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return -1;
}
```

aus IDE-Level 2.3 (Klasse Suche), ohne Zähler



Mitte ansehen

mid = (lo + hi) / 2 – ganzzahlig geteilt



Hälfte verwerfen

r[mid] < x: rechts weiter (lo = mid + 1), sonst links (hi = mid − 1)



Ende

Treffer → mid; lo > hi → −1

Voraussetzung: Die Reihung ist aufsteigend sortiert!

Wie beim Zahlenraten 1–100: immer die Mitte tippen.

Trace: lo, hi und mid Schritt für Schritt

Suche nach 67 in r (n = 12)

	0	1	2	3	4	5	6	7	8	9	10	11	
Durchlauf 1	**4**	**9**	**15**	**21**	**26**	**33**	**38**	**45**	**52**	**60**	**67**	**74**	**33 < 67 → lo = 6**
Durchlauf 2	4	9	15	21	26	33	**38**	**45**	**52**	**60**	**67**	**74**	**52 < 67 → lo = 9**
Durchlauf 3	4	9	15	21	26	33	38	45	52	**60**	**67**	**74**	**67 gefunden → 10**

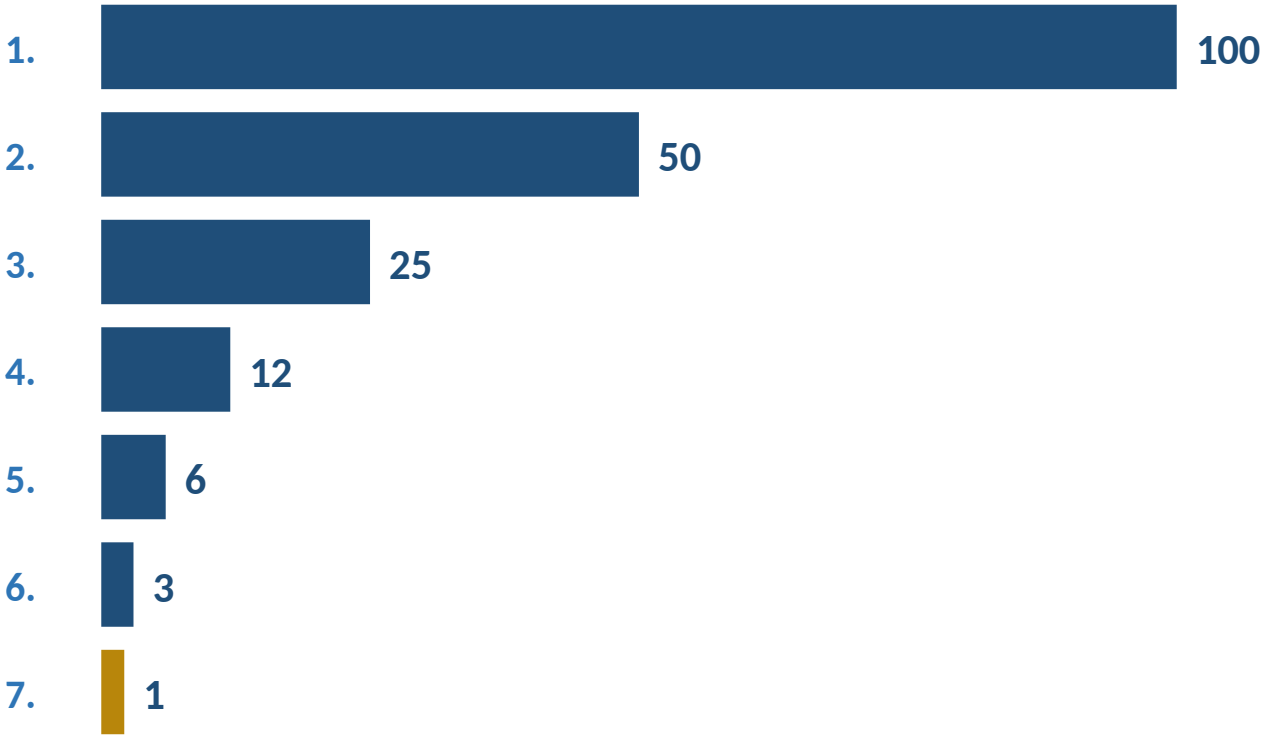
Suche nach 30 (nicht enthalten) als Tracetabelle

Durchlauf	lo	hi	mid	r[mid]	Aktion
1	0	11	5	33	hi ← 4
2	0	4	2	15	lo ← 3
3	3	4	3	21	lo ← 4
4	4	4	4	26	lo ← 5

lo = 5 > hi = 4
Schleife endet → gib −1 zurück. 4 Durchläufe = Maximum bei n = 12.

Wie oft kann man 100 halbieren?

Kandidaten vor jedem Durchlauf (schlechtester Fall, n = 100)



n = 100 → höchstens 7 Durchläufe, denn $2^6 = 64 \leq 100 < 128 = 2^7$

n	max. Durchläufe
100	7
200	8
1 000	10
2 000	11
1 000 000	20

n verdoppeln → + 1

Der erste zusätzliche Durchlauf halbiert 2n wieder auf n – danach geht es weiter wie vorher.

Linear oder binär?

Durchläufe im schlechtesten Fall

n = 100	linear 100	binär 7
n = 1 000	linear 1 000	binär 10
n = 1 000 000	linear 1 000 000	binär 20

Aber: binär nur auf sortierten Daten. Wer nur einmal sucht, sortiert nicht extra – Sortieren kostet bis zu n(n − 1)/2 Vergleiche.

AUSBLICK · NICHT ABITURRELEVANT

O-Notation

`O(log n)`	binäre Suche
`O(n)`	lineare Suche
`O(n log n)`	Mergesort (Welt 3)
`O(n²)`	Selection, Insertion, Bubble

Im Abitur wird gezählt: Vergleiche, schlechteste Eingabe, Verdopplung.

Abi-Format: Verfahren erkennen, Vergleiche bestimmen

AUFGABE (Beispiel) · 6 BE ≈ 15 min

Geben Sie an, welches Sortierverfahren die Methode geheim implementiert, und erläutern Sie seine Funktionsweise. Bestimmen Sie die Anzahl der Vergleiche r[j] < r[k] für 8 Elemente und beurteilen Sie, wie sie sich bei 16 Elementen ändert.

```
public void geheim(int[] r) {
    for (int i = 0; i < r.length - 1; i++) {
        int k = i;
        for (int j = i + 1; j < r.length; j++) {
            if (r[j] < r[k]) {
                k = j;
            }
        }
        int hilf = r[i];
        r[i] = r[k];
        r[k] = hilf;
    }
}
```

MUSTERANTWORT

Selectionsort (Auswahl des Minimums): Im Durchlauf i wird im unsortierten Rest r[i] … r[n − 1] der Index k des kleinsten Elements gesucht und dieses mit r[i] vertauscht. Danach ist r[0] … r[i] endgültig sortiert.

Durchlauf i vergleicht n − 1 − i Elemente, unabhängig von den Daten: 7 + 6 + … + 1 = 8 · 7 / 2 = 28 Vergleiche.

Bei 16 Elementen: 16 · 15 / 2 = 120, also etwa viermal so viele, weil die Anzahl quadratisch von n abhängt.

DAS GIBT PUNKTE

- Verfahren benennen
- Funktionsweise erläutern: Rest, Minimum, Tausch
- Anzahl mit Rechenweg
- Verdopplung begründen (quadratisch)

Boss 2: Das Turnier der Bogenschützen



DS 17

60 Minuten · ≈ 24 BE
Zeitbudget wie im Abi: ≈ 2,5 min pro BE

Papier, ohne Gerät
Tracen, Zählen, Implementieren – alles aus Welt 2

Neuer Kontext
keine Highscore-Liste – gleiche Bausteine, Transfer

Zusätzlich im Lab
Online-Boss „Das Wettfischen am Drachensee“ · 25 min

Vorbereitung: Lab-Level 2.1–2.4 · Trainingsplatz · Warm-up-Trainer · Tracetabellen · Abi-Check